OPENAI · DEVELOPER GUIDE

CODEX CLI
완벽 가이드

설치부터 엔터프라이즈 운영까지

터미널에서 만나는 OpenAI의 코딩 에이전트

참조 · developers.openai.com/codex

대상 모델 · GPT-5.4 / GPT-5.3-Codex / GPT-5.4-mini

OPENAI · DEVELOPER GUIDE

Codex CLI 완벽 가이드

설치부터 엔터프라이즈 운영까지

목 차

제1부 · 시작하기

01 Codex CLI 소개 9

02 Codex 모델 소개 13

03 설치와 인증 17

04 Quickstart — 첫 세션 만들기 21

05 CLI Cheatsheet 25

06 자주 사용하는 슬래시 명령어 31

제2부 · 세션 운영

01 파일 관리 — @ 멘션과 작업 디렉토리 35

02 셸 명령 실행 — ! 접두와 안전 모드 39

03 codex resume — 세션 재개 43

04 /fork — 세션 분기 47

05 /clear & /new — 컨텍스트 정리 49

06 /compact — 컨텍스트 압축 51

07 /status — 세션 상태 확인 53

08 AGENTS.md 55

09 Rules 59

제3부 · 작업 도구

01 Plan 모드 — 계획 수립 후 실행 63

02 /review — 4가지 프리셋 리뷰 67

03 /model — 모델 변경과 reasoning 단계 71

04 Approval 모드와 Sandboxing 75

05 /imagegen — 이미지 생성 79

제4부 · 확장과 자동화

01 Custom Slash Commands 81

02 Skills 85

03 MCP 89

04 Subagents 93

제5부 · 환경 설정

01 Config 파일 — Basic / Advanced / Reference 101

02 Authentication — 자격 증명 관리 107

03 Security — 보안 모델 이해 111

04 Enterprise 구성 — Admin과 Governance 115

05 Windows 환경 설정 119

제6부 · 운영과 활용

01 Pricing과 사용량 123

02 Non-interactive Mode와 Codex SDK 127

03 App Server / MCP Server 131

04 Troubleshooting 135

05 함께 쓸 때의 주의사항 139

06 마무리 — 강점과 활용 추천 143

PART 1 · CHAPTER 1

01

Codex CLI 소개

터미널에서 만나는
OpenAI의 코딩 에이전트

Codex CLI란?

Codex CLI는 OpenAI가 공개한 **로컬 터미널용 코딩 에이전트**입니다. `codex` 한 줄로 인터랙티브 TUI(Terminal UI)가 열리고, 작업 중인 디렉토리의 코드를 읽고 수정하고 명령을 실행하면서 사용자와 자연어로 대화합니다.

- **오픈소스** · GitHub `openai/codex`
- **Rust 기반** · 빠른 시작 시간과 낮은 메모리 사용량
- **npm 패키지** · `@openai/codex`
- **플랫폼** · macOS, Linux 정식 지원 / Windows 실험적(WSL 권장)
- **인증** · ChatGPT 계정 로그인 또는 OpenAI API 키
- **구독 플랜** · ChatGPT Plus, Pro, Business, Edu, Enterprise에 Codex 포함

왜 CLI 형태인가?

Codex는 웹 앱(ChatGPT), Codex Cloud, IDE 익스텐션 형태로도 제공되지만, CLI 버전은 다음 시나리오에 강합니다.

- SSH로 접속한 원격 서버에서 즉시 코딩 보조
- tmux·screen에 띄워 두고 다른 작업과 병행
- 셸 파이프와 결합 — `codex exec` 로 자동화 워크플로우 구성
- 개인 dotfiles · 환경변수 · 별칭(alias)과 자연스럽게 통합

핵심 동작 모델

구분	설명
읽기/쓰기	현재 작업 디렉토리(cwd)와 명시한 추가 경로 안에서 파일 조작
실행	Approval 모드(Auto / Read Only / Full Access)에 따라 셸 명령 실행
웹 검색	도구로 통합된 web search로 최신 정보 조회
이미지 입력	스크린샷·디자인 시안을 첨부하여 분석
MCP	외부 도구·문서·서비스를 플러그인처럼 연결
Sandbox	OS 네이티브 격리(macOS Seatbelt, Linux Landlock)로 안전 실행

Codex 제품군에서의 위치

- **Codex CLI** (이 책의 주제) · 터미널 네이티브, 가장 가볍고 빠름
- **Codex App** · 데스크톱 앱. 멀티 프로젝트·자동화 UI
- **Codex IDE Extension** · VS Code, Cursor 등 IDE 안에서 동작
- **Codex Cloud** · 격리된 클라우드 환경에서 비동기 실행
- **GitHub / Slack / Linear 통합** · 워크플로우 안에 직접 연결

> 💡 **정보 —** CLI에서 만든 MCP 서버, Skills, AGENTS.md, 인증 정보는 IDE 익스텐션과 App에서도 동일하게 사용됩니다. 한 번 설정하면 어디서든 같은 동료를 호출할 수 있습니다.

PART 1 · CHAPTER 2

02

Codex 모델 소개

CLI에서 만나는
최신 프런티어 모델 라인업

대표 모델 한눈에 보기

모델	특징	추천 용도
`gpt-5.4`	OpenAI 플래그십 프런티어 모델. 강력한 추론 + 코드 + 컴퓨터 사용 능력	대부분의 Codex 작업 (기본 권장)
`gpt-5.3-codex`	코드 특화 모델. 빠르고 정확한 편집·리팩터링	패치 단위 코드 수정
`gpt-5.4-mini`	속도와 비용을 줄인 소형 변종	대량 호출, 단순 작업
`gpt-5.3-codex-spark`	리서치 프리뷰. 응답이 매우 빠른 경량 변형 (Pro 전용)	가벼운 탐색·요약

Reasoning 단계 — 같은 모델, 다른 깊이

OpenAI의 reasoning 모델은 응답 전 내부적으로 "생각하는" 단계를 거칩니다. 이 깊이를 `minimal · low · medium · high`로 조정해 정확도와 속도·비용의 균형을 맞춥니다.

단계	특징	적합한 작업
`minimal`	거의 추론 없이 즉답	한 줄 답변, 간단 변환
`low`	가벼운 사고	코드 한 함수 작성, 짧은 설명
`medium`	균형 잡힌 기본값	일반적 코드 수정 (권장)
`high`	깊이 있는 추론	아키텍처 설계, 어려운 디버깅, 보안 리뷰

모델 선택 가이드

상황	추천 조합
설계·아키텍처 결정 · 어려운 디버깅	`gpt-5.4` + reasoning `high`
여러 파일에 걸친 패턴 리팩터링	`gpt-5.3-codex` + `medium`
탐색·요약·짧은 Q&A	`gpt-5.4-mini` + `low`
단순 명령 생성·보일러플레이트	`gpt-5.4-mini`

```
# 시작 시 모델 지정
codex --model gpt-5.4

# 세션 도중 변경 (모델 + reasoning)
/model
```

✅ **TIP —** 기본값으로 `gpt-5.4` + `medium`을 두고, 정말 어려운 문제에만 잠깐 `high`로 올리는 패턴을 권장합니다. 항상 `high`로 두면 비용과 응답 시간이 빠르게 늘어납니다.

PART 1 · CHAPTER 3

03

설치와 인증

npm 설치, ChatGPT 계정,
API 키

시스템 요구사항

- **OS** · macOS / Linux 정식 지원, Windows는 실험적(WSL 권장)
- **Node.js** · 최신 LTS
- **Shell** · Bash, Zsh, PowerShell

설치 방법

① npm (가장 일반적)

```
npm install -g @openai/codex
# 또는 약식
npm i -g @openai/codex
```

▸ ② Homebrew (macOS / Linux)

```
brew install codex
```

▸ ③ 업그레이드

```
npm i -g @openai/codex@latest
```

실행

```
codex                                  # 인터랙티브 실행
codex "이 코드베이스를 설명해줘"     # 초기 프롬프트와 함께 시작
codex --cd ./apps/web                  # 작업 루트 지정
codex --add-dir ../shared              # 추가 쓰기 경로 허용
codex exec "build script 작성"      # 비대화형 실행
```

인증

처음 `codex` 를 실행하면 인증 방법을 묻습니다. 두 가지 길이 있습니다.

▸ ① ChatGPT 계정 로그인 (권장)

1. `codex` 실행
2. "How would you like to authenticate?" → **Sign in with ChatGPT**
3. 브라우저에서 OpenAI 계정으로 로그인

ChatGPT Plus / Pro / Business / Edu / Enterprise 플랜에는 Codex 사용량이 포함되어 있어 별도 결제 없이 시작할 수 있습니다.

▸ ② OpenAI API 키

```
export OPENAI_API_KEY="sk-..."
codex
```

API 키 방식은 사용량당 과금되며, OpenAI 플랫폼 청구로 정산됩니다.

인증 정보가 저장되는 곳

- `~/.codex/auth.json` — 토큰·자격 증명
- `~/.codex/config.toml` — 사용자 설정
- `~/.codex/sessions/` — 세션 트랜스크립트

업데이트 확인

```
codex --version
# 새 버전이 있으면 업데이트
npm i -g @openai/codex@latest
```

> 💡 **정보 —** 엔터프라이즈 환경에서는 **Managed Configuration**으로 인증 방식을 강제할 수 있습니다(예: 개인 ChatGPT 로그인 금지, 회사 API 키만 허용). 5부 04장 참고.

PART 1 · CHAPTER 4

04

Quickstart

첫 세션 만들기

5분 안에 실용적인
한 가지 작업을 끝낸다

인터랙티브 모드의 기본 흐름

1. 프로젝트 디렉토리로 이동: `cd ~/projects/myapp`
2. `codex` 실행 → 인증 후 TUI 진입
3. 프롬프트 작성 → `Enter`
4. Codex가 계획을 보여주고 단계별 실행
5. 파일 편집·명령 실행 시 승인 요청 (Auto / Read Only / Full Access)
6. 변경 사항을 `/diff` 로 확인 후 git 커밋

예제 1 — 코드베이스 탐색

```
이 저장소의 구조를 한 페이지로 요약해줘.
- 핵심 폴더와 책임
- 데이터가 어디서 흐르는지
- 외부 의존성
```

Codex가 디렉토리 트리를 탐색하면서 핵심 파일을 읽고, 한 페이지 요약을 만들어 줍니다. 처음 들어가는 코드베이스에서 가장 시간을 아껴주는 패턴입니다.

예제 2 — 새 기능 구현

```
로그인 폼에 비밀번호 강도 표시기를 추가해줘.
- 클라이언트 측 검증으로
- 기존 디자인 시스템(@components/Button.tsx 참고)을 따라서
```

Codex가 관련 컴포넌트를 읽고 → 변경 계획을 제시 → 승인 후 파일 수정 → diff 표시 → 커밋 메시지 초안까지 한 흐름에 처리합니다.

예제 3 — 버그 추적

```
npm test 를 돌려서 실패하는 케이스를 확인하고
원인이 되는 함수를 찾아 고쳐줘.
- 수정 전후로 테스트가 통과하는지 다시 확인해줘.
```

예제 4 — 이미지 입력

```
codex -i mockup.png "이 디자인 그대로 React 컴포넌트로 만들어줘"
```

스크린샷이나 디자인 시안을 입력으로 전달하면, Codex가 시각 정보를 바탕으로 마크업과 스타일을 생성합니다.

유용한 단축 흐름

- `@` 입력 → 워크스페이스 파일 자동완성

- `!` 접두 → 로컬 셸 명령 실행 (예: `!ls -la`)
- `Esc` 두 번 → 직전 메시지 편집 후 그 지점에서 분기
- `Ctrl` + `G` → `$EDITOR` (vim 등)에서 긴 프롬프트 작성
- `/help` → 모든 슬래시 명령 표시
- `/quit` → 세션 종료

💡 **정보 —** 환경 설정(가상환경 활성화·환경변수 export·데몬 기동)은 **Codex 실행 전에** 마쳐 두는 편이 토큰을 가장 절약합니다. Codex가 매번 환경을 더듬어 찾을 필요가 없기 때문입니다.

✅ **TIP —** 처음에는 작은 프로젝트나 테스트 폴더에서 시작하세요. Codex가 어떤 권한을 요청하는지, 어떤 도구를 어떻게 조합하는지 관찰하면서 익숙해지는 게 안전합니다.

PART 1 · CHAPTER 5

05

CLI Cheatsheet

자주 쓰는 명령과 옵션을
한 장에 모았다

핵심 CLI 명령

명령	설명
`codex`	인터랙티브 TUI 시작
`codex "query"`	초기 프롬프트와 함께 인터랙티브 시작
`codex exec "query"`	비대화형 1회 실행 (자동화용)
`codex resume`	세션 피커로 재개
`codex resume --last`	가장 최근 세션 즉시 재개
`codex resume --all`	다른 디렉토리 세션까지 함께 보기
`codex fork`	저장된 세션 포크
`codex mcp`	MCP 서버 관리
`codex --version`	버전 확인

주요 CLI 옵션

옵션	설명
`--model <name>`	모델 지정 (gpt-5.4 / gpt-5.3-codex / gpt-5.4-mini)
`--cd <dir>`	작업 디렉토리 지정
`--add-dir <dir>`	추가 쓰기 가능 디렉토리
`-i, --image <path>`	이미지 첨부
`--ask-for-approval <mode>`	Approval 모드 사전 지정
`--full-auto`	모든 도구 자동 승인 (= Auto 모드)
`--dangerously-bypass-approvals-and-sandbox`	(이전 `--yolo`) 위험. 신뢰된 환경에서만
`--sandbox <mode>`	샌드박스 정책 (read-only / workspace-write / danger-full-access)
`--config key=value`	config.toml 일회성 오버라이드

인터랙티브 핵심 단축키

단축키	동작
Enter	메시지 전송
Shift + Enter	줄바꿈
↑ / ↓	드래프트 히스토리 이동
Ctrl + R	프롬프트 히스토리 검색
Ctrl + C	입력 비우기 / 작업 중단 / 세션 종료(연속)
Ctrl + L	화면 정리(대화 유지)
Ctrl + G	외부 에디터에서 긴 프롬프트 작성
Ctrl + O	가장 최근 응답 클립보드 복사
Esc ×2	특정 메시지로 돌아가서 편집·분기
Tab	실행 중에 다음 턴 프롬프트 큐에 추가

입력 접두사

접두사	의미
`/`	슬래시 명령 — CLI 메타 제어 (다음 장 참고)
`@<path>`	파일·폴더를 컨텍스트에 첨부, 자동완성 지원
`!<cmd>`	로컬 셸 명령 실행 (사용자 권한)

✅ **TIP** — 옵션 이름이 헷갈릴 때는 `codex --help` 가 가장 빠른 길입니다. 인터랙티브 안에서는 `/help` 가 같은 역할을 합니다.

PART 1 · CHAPTER 6

06

자주 사용하는 슬래시 명령어

TUI 안에서 가장 자주 쓰이는
명령들을 한 자리에

슬래시 명령이란?

슬래시 명령(`/`로 시작)은 **대화 메시지가 아니라 CLI 자체에 보내는 메타 명령**입니다. 모델 호출 없이 즉시 처리되며, 세션·모델·승인·확장을 제어합니다. 모든 명령을 보고 싶다면 `/help`를 입력하세요.

① 세션 흐름 — 가장 자주 쓰는 핵심

명령	역할
`/help`	전체 슬래시 명령 도움말
`/clear`	화면 + 대화 모두 정리, 새 대화 시작
`/new`	화면 유지, 컨텍스트만 새로 시작
`/quit` 또는 `/exit`	세션 종료 (자동 저장)
`/copy`	가장 최근 응답을 시스템 클립보드로 복사 (= Ctrl + O)
`/diff`	현재 워킹 트리의 git 변경사항 표시

② 컨텍스트와 토큰 관리

명령	역할
`/compact`	대화를 요약으로 대체해 컨텍스트 토큰 회수
`/status`	현재 세션의 모델·승인·토큰·MCP 상태
`/statusline`	푸터에 표시할 항목 커스터마이즈

```
# 대화가 길어지고 응답이 느려지면
/compact

# 비용·환경이 의심스러우면 한 번씩
/status
```

③ 세션 분기와 재개

명령	역할
`/fork`	현재 대화를 새 스레드로 복제 — 다른 길 시도
Esc ×2	특정 메시지로 돌아가서 거기부터 다시 (편집 분기)

세션 자체를 재개하는 건 셸에서: `codex resume`, `codex resume --last`, `codex resume --all`.

④ 모델과 모드 전환

명령	역할
`/model`	현재 모델 + reasoning 단계 변경 다이얼로그
`/plan`	Plan 모드(읽기 전용) 진입
`/review`	로컬 코드 리뷰 4가지 프리셋 (base / uncommitted / commit / custom)
`/permissions` 또는 `/approvals`	승인 모드 변경 (Auto / Read Only / Full Access)
`/imagegen`	이미지 생성 (스킬로 패키징된 형태)

⑤ 확장 관리 — MCP / Skills

명령	역할
`/mcp`	활성 MCP 서버와 도구 목록
`/skills`	로드된 스킬 목록과 호출 메뉴
`/agent` (= `/subagents`)	활성 서브에이전트 스레드 전환·검사

⑥ 진단·기타

명령	역할
`/debug-config`	설정 레이어(전역·프로젝트·CLI) 우선순위 진단
`/init`	현재 디렉토리 분석 후 AGENTS.md 초안 생성
`/sandbox-add-read-dir <path>`	(Windows) 샌드박스에 읽기 권한 추가

실무에서 가장 많이 쓰는 10개

처음에는 다음 10개만 외우면 90%의 작업은 무리 없이 흘러갑니다.

1. `/help` — 다른 명령이 기억나지 않을 때
2. `/quit` — 종료 (자동 저장됨)
3. `/clear` — 화면과 대화 모두 정리
4. `/copy` — 마지막 응답 복사
5. `/compact` — 대화가 길어졌을 때
6. `/status` — 현재 환경 점검
7. `/model` — 모델·reasoning 전환
8. `/plan` — 큰 변경 전 계획 모드로
9. `/review` — 커밋 직전 리뷰
10. `/fork` — 다른 접근을 시도하기 전 안전망

> 💡 **정보 —** 슬래시 명령은 자동완성됩니다. `/` 만 입력해도 메뉴가 뜨고, 일부 글자만 입력해도 가장 가까운 후보가 제안됩니다.

PART 2 · CHAPTER 1

01

파일 관리

@ 멘션과 작업 디렉토리

Codex가 손댈 수 있는 파일을
정확히 한정한다

워크스페이스의 개념

Codex CLI는 시작한 디렉토리(cwd)와 그 하위를 기본 워크스페이스로 봅니다. 그 밖의 위치는 **읽기·쓰기 모두 거부**됩니다(샌드박스 정책으로 강제). 다른 폴더가 필요하면 명시적으로 추가해야 합니다.

```
# 시작 시 추가 디렉토리 허용
codex --add-dir ../shared --add-dir ../config

# 작업 루트 자체를 다른 곳으로
codex --cd ./apps/web
```

@ 멘션으로 파일 첨부

프롬프트에서 `@`를 입력하면 워크스페이스의 파일과 폴더가 자동완성으로 떠오릅니다. 선택한 항목의 내용이 모델 입력에 포함됩니다.

```
@src/auth/login.ts 의 로그인 로직을 리뷰해줘.
@docs/spec.md 와 비교해서 누락된 케이스를 찾아줘.
```

- 파일 한 개, 여러 개, 폴더 통째로 모두 가능
- 자동완성은 fuzzy matching
- git이 무시하는 파일(`.gitignore`)은 기본적으로 첨부 후보에서 제외
- `.codexignore`에 패턴 추가로 별도 제외 가능

승인 모드와 파일 변경

Codex가 파일을 새로 만들거나 수정할 때 권한을 묻는 정도는 Approval 모드로 결정됩니다.

모드	읽기	쓰기	실행
Read Only	✅	❌	❌
Auto (기본)	✅	워크스페이스 내	워크스페이스 내
Full Access	✅	어디든	어디든

실전 패턴

- 처음 보는 코드베이스 → **Read Only**로 안전 탐색
- 익숙한 코드의 반복 작업 → **Auto**로 흐름 유지
- npm install·도커 빌드 등 잠시만 → **Full Access** 후 즉시 복귀

git과의 관계

Codex는 git을 자연스럽게 인식합니다.

- `/diff` 로 변경사항 확인
- 커밋 메시지 초안 자동 작성
- `.gitignore` 존중
- `git stash` · `git checkout` 도 셸 명령으로 자유 실행

> ⚠️ **주의 —** Codex는 워크스페이스 밖 경로에 절대 쓰지 않습니다(샌드박스가 OS 수준에서 차단). `/etc/` 나 홈 디렉토리에 변경이 필요하면 직접 그 폴더에서 시작하거나 `--add-dir` 로 명시적으로 허용해야 합니다.

PART 2 · CHAPTER 2

02

셸 명령 실행

! 접두와 안전 모드

에이전트가 명령을 직접 실행하되
경계는 분명하게

두 가지 실행 경로

▸ ① 사용자가 직접 실행 — ! 접두

프롬프트 첫 글자가 `!`이면 그 라인 전체가 **사용자의 셸 명령**으로 해석됩니다. Codex는 출력을 컨텍스트에 추가하지만 자체적으로 실행하진 않습니다.

```
!ls -la
!git status
!npm test
```

▸ ② Codex가 실행 — 자연어 요청

"테스트 돌려줘", "lint 검사해줘" 같은 자연어 요청에서는 Codex가 적절한 명령을 골라 실행 권한을 요청합니다.

```
npm test 를 돌리고 실패하는 케이스만 알려줘.
```

승인 흐름

Codex가 명령을 실행하기 전에 다음과 같은 선택지를 제시합니다.

- **Approve** · 이번 한 번만 허용
- **Approve always for this command** · 같은 명령은 다음부터 자동
- **Reject** · 거부, 다른 방법을 찾도록 지시
- **Edit** · 명령을 수정해서 실행

샌드박스와 결합

Auto 모드의 워크스페이스 격리는 OS 네이티브 기술로 강제됩니다.

- **macOS** · Seatbelt 프로파일
- **Linux** · Landlock LSM
- **네트워크** · 기본 차단, 명시적 허용 필요

승인 사전 지정

```
# 모든 도구 자동 승인 (Auto 모드 = full-auto)
codex --full-auto

# 모드 사전 지정
codex --ask-for-approval read-only
codex --ask-for-approval workspace-write
codex --ask-for-approval danger-full-access
```

실전 패턴

상황	추천 모드
모르는 저장소 탐색	Read Only
일상 개발	Auto (기본)
도커·시스템 명령 잠깐 필요	Full Access (작업 후 즉시 복귀)
CI / 비대화형	`codex exec --full-auto`

⚠️ **주의 —** Full Access(이전 `--yolo`) 모드는 머신 어디든 쓰고 네트워크에도 자유롭게 접근합니다. 신뢰할 수 있는 저장소·작업에만 일시적으로 사용하고, 신뢰할 수 없는 코드는 컨테이너나 별도 VM에서 실행하세요.

PART 2 · CHAPTER 3

03

codex resume

세션 재개

예전 대화의 컨텍스트를
그대로 이어서 작업하기

세션은 자동 저장됩니다

Codex는 모든 인터랙티브 세션을 `~/.codex/sessions/` 아래에 저장합니다. 종료한 뒤에도 같은 작업 디렉토리에서 다시 열 수 있습니다.

세 가지 호출 방식

① 피커에서 고르기

```
codex resume
```

최근 인터랙티브 세션 목록이 표시되고, 항목을 강조하면 요약이 나옵니다. `Enter` 로 재개합니다.

▸ ② 모든 세션 보기 (`--all`)

```
codex resume --all
```

현재 작업 디렉토리 외의 다른 디렉토리에서 시작했던 세션까지 함께 보여 줍니다.

▸ ③ 가장 최근 세션 즉시 재개 (`--last`)

```
codex resume --last
codex resume --last --all   # 디렉토리 필터 무시하고 가장 최근
```

▸ ④ 특정 세션 ID 지정

```
codex resume 7f9f9a2e-1b3c-4c7a-9b0e-........
```

세션 ID는 피커에서 복사하거나, `/status` 출력 또는 `~/.codex/sessions/` 파일명에서 확인할 수 있습니다.

비대화형 자동화에서도 재개 가능

```
codex exec resume --last "찾은 race condition을 고쳐줘"
codex exec resume 7f9f9a2e... "계획 그대로 구현해줘"
```

재개 시 보존되는 것

- 원본 트랜스크립트 (모든 메시지)
- 계획 히스토리
- 승인 기록
- 활성 모델·reasoning 단계

덮어쓰기 가능한 항목

```
codex resume --last --cd ../another-app
codex resume --last --add-dir ../shared
codex resume --last --model gpt-5.4
```

💡 **정보 —** resume 직후 가장 먼저 `/status` 를 실행해 의도한 환경(모델·승인·MCP)이 그대로 떠 있는지 확인하는 습관을 들이세요. 며칠 만에 돌아온 세션에서 잘못된 모드로 작업하는 사고를 막아 줍니다.

PART 2 · CHAPTER 4

04

/fork

세션 분기

하나의 대화에서
두 가지 길을 동시에 탐색

왜 포크가 필요한가?

"이 함수를 리팩터링하는 두 가지 방법을 모두 보고 싶다", "현재 계획을 망치지 않으면서 다른 접근을 시도해 보고 싶다" — 이런 순간에 `/fork`를 씁니다. 현재 트랜스크립트를 **새 스레드로 복제**해 원본은 그대로 둡니다.

사용법

```
/fork
```

현재 대화가 새 ID의 스레드로 복제되어 그쪽으로 자동 전환됩니다. 원본 세션은 `codex resume`으로 언제든 다시 열 수 있습니다.

저장된 세션을 포크하기

```
codex fork
# 세션 피커가 열림 → 분기할 세션 선택
```

관련 흐름과의 차이

명령	역할
`/fork`	현재 대화 복제 → 새 스레드에서 다른 시도
`/new`	같은 CLI 세션 안에서 컨텍스트만 새로 시작 (히스토리 유지)
`/clear`	화면도 비우고 새 대화 시작
Esc ×2	특정 메시지로 돌아가서 거기서부터 다시 (편집 후 분기)

전형적인 활용

- **대안 비교** · 같은 문제를 두 가지 방식으로 풀어 결과 비교
- **위험한 시도 격리** · 큰 리팩터링 전에 안전망
- **학습 분기** · 같은 코드를 여러 모델로 분석시켜 비교
- **실험 보존** · 잘 된 분기는 살리고 실패한 분기는 버림

✅ **TIP —** 큰 리팩터링 전에는 항상 `/fork` 로 안전망을 만든 뒤 시도하세요. 잘못되어도 원본 흐름은 그대로 남아 있어 비교가 쉽습니다.

PART 2 · CHAPTER 5

05

/clear & /new

컨텍스트 정리

화면도, 대화도
새로 시작하기

/clear의 동작

`/clear` 는 다음을 한 번에 수행합니다.

- 터미널 화면을 깨끗이 지움
- 현재 대화 컨텍스트를 비우고 **새 대화** 시작
- 같은 CLI 세션 내에서 머무르므로 인증·MCP 서버 연결은 그대로 유지

/new의 동작

`/new` 는 화면은 그대로 두고 컨텍스트만 새로 시작합니다. 기존 대화의 시각적 흐름을 보존하고 싶을 때 유용합니다.

유사 명령과 비교

명령/단축키	화면	대화	비고
`/clear`	지움	새로 시작	가장 깨끗한 리셋
`/new`	유지	새로 시작	화면 흐름 보존
Ctrl + L	지움	유지	화면만 정리
`/compact`	유지	요약 압축	장기 작업 유지
`/fork`	유지	새 스레드	분기 비교
`/quit`	종료	저장 후 종료	완전히 끝

언제 어느 명령을 쓸까?

- 한 작업이 끝났고 다음으로 완전히 넘어갈 때 → `/clear`
- 같은 주제의 다음 단계인데 컨텍스트를 가볍게 → `/new`
- 지금까지 흐름을 살리되 토큰만 회수 → `/compact`
- 다른 접근을 비교하면서 시도 → `/fork`

> ⚠️ **주의 —** Codex가 한창 작업 중일 때는 `/clear` 와 Ctrl + L 이 모두 비활성화됩니다. Ctrl + C 로 작업을 먼저 중단해야 합니다.

PART 2 · CHAPTER 6

06

/compact

컨텍스트 압축

긴 대화를 통째로 버리지 않고
요약해서 자리 비우기

왜 압축인가?

대화가 길어지면 컨텍스트 한도가 가까워집니다. `/clear`로 통째로 비우면 흐름까지 잃어버리고, 그대로 두면 조만간 잘립니다. `/compact`는 **이전 턴들을 핵심만 남긴 요약으로 대체**해 컨텍스트 토큰을 회수합니다.

사용법

```
/compact
```

1. Codex가 대화를 요약해도 되는지 확인
2. 승인하면 이전 턴이 요약 블록으로 대체
3. 핵심 결정·합의·중요한 코드 조각은 보존

요약에 보존되는 것

- 핵심 결정과 그 근거
- 완료한 작업 목록
- 발견된 사실(파일 위치, 함수 시그니처 등)
- 아직 미완료인 todo
- 사용자의 명시적 지시·선호

언제 트리거할까?

- `/statusline`의 컨텍스트 사용량이 70~80%를 넘기 시작할 때
- 한 작업이 끝나고 다음 작업으로 넘어가는 자연스러운 매듭
- 같은 세션에서 여러 PR을 연속으로 처리할 때 PR 사이

관련 도구

명령	역할
`/compact`	요약하여 토큰 회수 (흐름 유지)
`/clear`	완전히 비움 (흐름 단절)
`/new`	같은 CLI 안에서 새 대화
`codex resume`	나중에 원본 흐름으로 돌아가기

> 💡 **정보 —** `/compact`가 만든 요약은 그 자체로 좋은 회의록이 됩니다. 필요하면 `/copy`로 가져와서 `NOTES.md` 같은 파일에 보관해도 좋습니다.

PART 2 · CHAPTER 7

07

/status

세션 상태 확인

현재 세션이 어떤 환경에서
돌고 있는지 한 눈에

표시되는 정보

```
/status
```

- 활성 모델 + reasoning 단계
- Approval 모드
- 쓰기 가능 경로(writable roots)
- 현재 토큰 사용량 / 남은 컨텍스트
- 세션 ID (resume에 사용)
- 활성 MCP 서버 / Skills / Subagents

언제 쓰는가?

- 긴 작업 전에 **의도한 모델로 잘 떠 있는지** 확인
- `/compact` 타이밍 판단
- 승인 정책이 `Auto` 인지 `Read Only` 인지 헷갈릴 때
- MCP 서버가 정상적으로 붙었는지 점검

관련 진단 명령

명령	역할
`/status`	현재 세션 상태 한 눈에
`/debug-config`	설정 레이어(전역·프로젝트·CLI 플래그) 우선순위 진단
`/mcp`	활성 MCP 서버와 도구 목록
`/skills`	로드된 스킬 목록
`/diff`	git 변경사항 (스테이지·언스테이지·미추적 포함)

statusline과의 결합

`/statusline` 으로 푸터에 자주 보고 싶은 항목을 미리 띄워 두면 `/status` 를 매번 호출하지 않아도 됩니다.

```
/statusline
# 추천 조합: 컨텍스트 사용량 + git 브랜치 + 모델
```

✅ **TIP —** 셸에서 `codex status` 를 실행해도 같은 정보를 얻을 수 있습니다. CI/스크립트에서 점검할 때 유용합니다.

PART 2 · CHAPTER 8

08

AGENTS.md

프로젝트 컨텍스트 계층

프로젝트 규칙을
모델 시스템 프롬프트에 연결

AGENTS.md란?

`AGENTS.md` 는 프로젝트의 규칙·아키텍처·관습을 적어 두는 **표준 컨텍스트 파일**입니다. Codex는 시작 시 디렉토리 트리를 따라 올라가며 모든 `AGENTS.md` 를 수집해 시스템 프롬프트에 합칩니다.

위치별 역할

위치	스코프
`~/.codex/AGENTS.md`	전역 — 모든 프로젝트 공통 사용자 선호
`<repo>/AGENTS.md`	저장소 루트 — 팀 공유 규칙
`<repo>/<subdir>/AGENTS.md`	하위 디렉토리 — 모듈별 추가 규칙

로딩 순서

1. 전역 → 저장소 루트 → 하위 디렉토리 순으로 누적
2. 가까운 위치(하위 디렉토리)일수록 우선순위가 높음
3. 동일 항목이 충돌하면 가까운 것이 이김

잘 작성된 AGENTS.md 예시

```
# 프로젝트: payments-service

## 아키텍처
- DDD + Hexagonal Architecture
- 도메인 로직은 src/domain/ — 외부 의존 0
- src/adapters/ 만 외부 시스템(DB, 메시지 큐)에 의존

## 코드 스타일
- TypeScript strict 모드
- 함수형 우선, 클래스는 도메인 모델에만
- import는 절대 경로 (`@/domain/...`)

## 테스트
- Vitest. 단위 테스트는 같은 폴더에 *.test.ts
- 통합 테스트는 tests/integration/

## 빌드/실행
- `pnpm dev` — 개발 모드
- `pnpm test` — 전체 테스트

## 절대 하지 말 것
- src/legacy/ 수정 (조만간 폐기)
- DB 마이그레이션 자동 실행 (사람 검토 필수)
```

다른 파일 임포트

```
@./docs/architecture.md
@./docs/api-conventions.md
@~/configs/team-style.md
```

`@` 임포트로 다른 마크다운 파일의 내용을 끌어올 수 있습니다. 큰 문서를 분할 관리할 때 유용합니다.

/init으로 초안 자동 생성

```
/init
```

현재 디렉토리의 README, package.json, 폴더 구조를 분석해 적절한 AGENTS.md 초안을 만들어 줍니다. 새 프로젝트에서 가장 시간을 아껴주는 명령입니다.

> ✅ **TIP** — 처음에는 50줄을 넘기지 말고 가장 자주 어기는 규칙만 적으세요. 나중에 "Codex가 왜 이걸 또 모르지?" 싶은 일이 반복되면 그때 항목을 추가하면 됩니다.

PART 2 · CHAPTER 9

09

Rules

동작 규칙 정의

에이전트가 반드시 지켜야 할 선을
강제하기

AGENTS.md vs Rules

둘은 비슷해 보이지만 역할이 다릅니다.

항목	AGENTS.md	Rules
형식	마크다운 — 자유로운 설명	구조화된 설정 — 강제 가능
역할	프로젝트 컨텍스트 — 알리기	강제 규칙 — 막기/허용
전달 경로	시스템 프롬프트에 합쳐짐	실행 시점에 정책 엔진이 평가
예시	"함수형 우선"	"shell `rm -rf` 는 항상 거부"

Rules의 형태

Rules는 `~/.codex/config.toml` 또는 프로젝트 `.codex/config.toml`에 정의합니다.

```
# ~/.codex/config.toml

[[rules]]
tool = "shell"
match = "^rm -rf"
action = "deny"

[[rules]]
tool = "shell"
match = "^npm (test|run lint)"
action = "allow"

[[rules]]
tool = "fetch"
match = "^https?://(?!.*\\.company\\.com)"
action = "deny"
reason = "외부 네트워크 접근 차단"
```

액션 종류

- **allow** · 매칭되면 자동 승인 (사용자 확인 생략)
- **deny** · 매칭되면 즉시 거부 (사용자 확인 우회)
- **approve** · 자동 거부되었을 일을 사용자 승인 대상으로 격상

왜 Rules가 필요한가?

- **안전 가드레일** · 위험한 명령을 모델 의지와 무관하게 막음
- **마찰 줄이기** · 안전한 반복 명령은 자동 승인해 흐름 유지
- **회사 정책 강제** · 외부 도메인 차단, 특정 도구만 허용
- **비대화형 실행** · CI에서 사용자 승인 없이 안전 운영

전형적인 규칙 세트

```
# 일상 개발용

[[rules]]
tool = "shell"
match = "^(ls|pwd|cat|grep|find|git status|git diff)"
action = "allow"

[[rules]]
tool = "shell"
match = "^(npm test|pnpm test|yarn test)"
action = "allow"

[[rules]]
tool = "shell"
match = "^(rm -rf|sudo|chmod 777)"
action = "deny"

[[rules]]
tool = "shell"
match = "^git push"
action = "approve"   # 자동 안 됨, 매번 사람 승인
```

Policy Engine과의 관계

Codex의 Rules는 더 큰 **Policy Engine**의 일부입니다. Policy Engine은 도구·인자·환경(예: cwd, 시간대)을 모두 평가해 정책을 적용합니다. 엔터프라이즈 환경에서 Managed Configuration으로 강제할 수 있습니다(다음 부 참고).

> ⚠️ **주의 —** Rules는 모델보다 위에서 동작합니다. 사용자가 자연어로 "rm -rf 해줘"라고 해도 deny 규칙이 있으면 차단됩니다 — 이게 Rules의 핵심 가치입니다. 다만 너무 많이 deny하면 모델이 다른 우회 경로를 시도해 컨텍스트를 낭비할 수 있으니, 정말 위험한 것만 막으세요.

PART 3 · CHAPTER 1

01

Plan 모드

계획 수립 후 실행

코드를 만지기 전에
설계도부터 받아낸다

왜 plan 모드가 필요한가?

큰 변경 — 새 기능 구현, 마이그레이션, 다중 파일 리팩터링 — 을 곧바로 시키면 Codex가 잘못된 가정 위에서 코드를 쓰기 시작할 수 있습니다. `/plan`은 작업 전에 **실행 계획을 먼저 제안**받는 안전 장치입니다.

사용법

```
/plan
/plan 이 서비스를 NestJS로 마이그레이션할 계획을 세워줘
```

인라인 프롬프트와 함께 이미지·파일을 첨부할 수도 있습니다.

전형적인 흐름

1. `/plan` 으로 진입
2. Codex가 단계별 계획을 제시 (탐색 → 설계 → 구현 → 검증)
3. 계획을 검토하면서 수정 요청 ("3단계는 빼자", "테스트도 함께")
4. 준비되면 일반 모드로 돌아와 계획에 따라 구현 지시

Plan 모드의 출력 형식

Plan 모드에서 모델은 다음 형식으로 답변하도록 유도됩니다.

- **현재 상태(Discovery)** — 무엇을 읽었고 어떤 가정을 했는지
- **의도(Intent)** — 무엇을 하려는지
- **단계(Steps)** — 실제로 변경할 파일·명령
- **위험(Risks)** — 알려진 함정·되돌리기 어려운 작업
- **검증(Validation)** — 어떻게 성공을 확인할지

실전 예시 — 마이그레이션 계획

```
/plan
이 Express 서비스를 NestJS로 옮기는 계획을 세워줘.
- DB 스키마는 그대로 유지
- 기존 라우트의 외부 API 호환성 유지
- 단계별로 의존성·실행 명령·검증 방법을 명시해줘
```

모델 조합 패턴

계획 단계는 추론이 깊어야 하고, 실행 단계는 빠른 반복이 중요합니다. 다음과 같이 모델을 교차해 쓰면 비용·품질이 모두 좋아집니다.

```
# 1단계 - Plan은 강한 모델로
/model gpt-5.4 high
/plan ...

# 2단계 - 실행은 코드 특화로
/model gpt-5.3-codex medium
"위 계획대로 진행해줘"
```

⚠️ **주의 —** 이미 다른 작업이 진행 중일 때는 `/plan` 이 일시적으로 잠깁니다. `Ctrl` + `C` 로 멈춘 뒤에 다시 시도하세요.

💡 **정보 —** Plan 모드 결과를 **그대로 별도 마크다운으로 저장**해 두면, 같은 작업의 두 번째 시도(다른 모델/접근)에서 좋은 비교 기준이 됩니다.

PART 3 · CHAPTER 2

02

/review

4가지 프리셋 리뷰

"내 코드 한 번 봐줘"를
4가지 범위로

사용법

```
/review
```

프리셋을 고르는 팝업이 뜹니다. 어떤 프리셋이든 **워킹 트리는 건드리지 않고**, 발견 사항만 리포트합니다.

4가지 프리셋

프리셋	대상	활용 시점
base	로컬 브랜치를 고르면 그 upstream과의 merge-base에서부터의 diff	PR 올리기 직전, 최종 점검
uncommitted	스테이지 + 언스테이지 + 미추적 파일	커밋 직전
commit	최근 커밋 목록에서 SHA 선택	특정 변경 한 건만 정밀 검토
custom	사용자가 직접 지시문 입력	"접근성 회귀에 집중" 같은 특화 검토

리포트 형식

- 각 발견 사항마다 **우선순위 · 근거 · 재현 단계**
- 스타일 코멘트는 줄이고, 진짜 버그·회귀를 우선 노출
- 같은 변경을 다시 검토하면 새 턴으로 기록되어 비교 가능
- 리뷰는 **읽기 전용** — 코드를 직접 고치지 않음

리뷰 모델 지정

```
# ~/.codex/config.toml
review_model = "gpt-5.4"
```

지정하지 않으면 현재 세션 모델을 그대로 사용합니다. **리뷰 전용으로는 더 높은 정확도의 모델**을 두는 패턴이 흔합니다.

Auto Review

변경이 일어날 때마다 자동으로 별도 리뷰어 에이전트가 돌도록 설정할 수 있습니다.

```
# ~/.codex/config.toml
[review]
auto = true
preset = "uncommitted"
```

CI에서 활용

```
# PR 코멘트로 사용
codex exec "/review base" \
  --output-format json \
  > review.json
```

✅ **TIP —** 리뷰가 끝나면 발견 사항을 보고 다시 Codex에 수정을 지시하는 흐름이 안전합니다. "리뷰가 지적한 1, 3, 5번 항목을 고쳐줘" 같은 식으로요.

PART 3 · CHAPTER 3

03

/model

모델 변경과 reasoning 단계

한 세션 안에서
여러 모델을 자유롭게 교차

기본 사용

```
/model
```

팝업에서 모델과 reasoning 단계를 함께 선택합니다. 변경 후 트랜스크립트에 확인 메시지가 출력됩니다.

설정 우선순위

모델은 다음 순서로 결정됩니다(높은 것이 이김).

1. `--model` CLI 옵션
2. 세션 도중 `/model` 변경
3. `~/.codex/config.toml` 의 `model` 키

4. 기본값 (`gpt-5.4`)

config.toml에서 영구 설정

```
# ~/.codex/config.toml
model = "gpt-5.4"
model_reasoning_effort = "medium"

# 리뷰 전용 모델
review_model = "gpt-5.4"
```

커스텀 에이전트 단위로 지정

서브에이전트나 스킬마다 다른 모델·reasoning을 쓸 수 있습니다.

```
# .codex/agents/reviewer.toml
name = "reviewer"
model = "gpt-5.4"
model_reasoning_effort = "high"
sandbox_mode = "read-only"
```

실전 전략

상황	추천 조합
계획·아키텍처	`gpt-5.4` + `high`
여러 파일 리팩터링	`gpt-5.3-codex` + `medium`
탐색·요약·짧은 Q&A	`gpt-5.4-mini` + `low`
보일러플레이트 대량 생성	`gpt-5.4-mini` + `minimal`
보안·위험 코드 리뷰	`gpt-5.4` + `high`

전환 패턴 — 한 작업 안에서 모델 교차

```
# 1. 처음에는 강한 모델로 계획
/model gpt-5.4 high
/plan 이 모듈을 함수형으로 다시 짤 계획

# 2. 실행 단계는 코드 특화로
/model gpt-5.3-codex medium
"위 계획대로 src/legacy/orders.ts 부터 진행해줘"

# 3. 사후 검토는 다시 강한 모델로
/model gpt-5.4 high
/review uncommitted
```

💡 **정보 —** 변경 후 `/status` 를 실행하면 활성 모델·reasoning·승인·MCP를 한 번에 확인할 수 있습니다. 비용이 갑자기 늘었다면 `high` 가 켜진 채로 잊혀진 경우가 흔합니다.

PART 3 · CHAPTER 4

04

Approval 모드와 Sandboxing

에이전트가 묻지 않고 할 수 있는 일을
어디까지 허용할지

승인 모드 세 가지

모드	읽기	쓰기	명령 실행	네트워크
Read Only	✅	❌	❌	❌
Auto (기본)	✅	✅ 워크스페이스	✅ 워크스페이스	❓ 묻고 사용
Full Access	✅	✅ 어디든	✅ 어디든	✅ 자동

모드 변경

```
/permissions
# 또는 별칭
/approvals
```

팝업에서 모드를 선택하면 그 시점부터 즉시 적용됩니다. 다음에 다시 바꿀 때까지 유지됩니다.

시작 시 모드 사전 지정

```
codex --ask-for-approval read-only
codex --ask-for-approval workspace-write
codex --ask-for-approval danger-full-access
codex --full-auto                # = Auto와 동등
```

샌드박스 — OS 수준 격리

Auto 모드에서 워크스페이스 격리는 **OS 네이티브 기술**로 강제됩니다. 모델이 워크스페이스 밖에 쓰려고 시도해도 시스템이 거부합니다.

- **macOS** · Seatbelt 프로파일로 파일·네트워크 차단
- **Linux** · Landlock LSM + seccomp으로 syscall 단위 차단
- **Windows** · 실험적 — WSL 사용 권장

샌드박스 정책 직접 지정

```
codex --sandbox read-only
codex --sandbox workspace-write
codex --sandbox danger-full-access
```

실전 워크플로우

1. 처음 보는 코드베이스 → **Read Only**로 탐색·계획
2. 계획 승인 후 → **Auto**로 작업

3. 도커 빌드·시스템 설치 등 잠시만 → **Full Access**
4. 끝나면 즉시 **Auto**로 복귀

Windows 추가 옵션

```
/sandbox-add-read-dir C:\absolute\directory\path
```

Windows 네이티브 CLI에서, 샌드박스 정책을 새로고침해 특정 디렉토리에 읽기 권한을 추가합니다.

> ⚠️ **주의 —** Full Access(이전 `--yolo`) 모드는 머신 어디든 쓰고 네트워크에도 자유롭게 접근합니다. 신뢰할 수 있는 저장소·작업에만 일시적으로 사용하세요. 가장 안전한 자동화는 **컨테이너 안에서 Full Access**로 도는 것입니다.

PART 3 · CHAPTER 5

05

/imagegen

이미지 생성

CLI 안에서 바로
로고·일러스트·시안 만들기

개요

Codex CLI는 입력으로 이미지를 받을 뿐 아니라, 슬래시 또는 스킬을 통해 **이미지 생성**도 호출할 수 있습니다. 작업 중인 프로젝트의 README 배지, 로고, 데모 스크린샷을 그 자리에서 만들 수 있습니다.

이미지 입력

```
codex -i screenshot.png "이 에러를 설명해줘"
codex --image img1.png,img2.jpg "두 다이어그램을 비교해줘"
```

인터랙티브 컴포저에 이미지를 붙여넣을 수도 있습니다.

이미지 생성 호출

이미지 생성은 보통 **스킬(Skill)** 형태로 패키징되어 있고, `$` 멘션이나 슬래시 명령으로 호출합니다.

```
$imagegen 미니멀 SVG 스타일로 새 모티브 로고를 그려줘.
주조색은 청록색, 정사각 1:1 캔버스로 만들어줘.
```

실전 활용 아이디어

- 오픈소스 프로젝트 로고 시안 여러 장 생성
- 블로그 글의 thumbnail 자동 생성
- UI 와이어프레임을 입력으로 주고 **고해상도 시안** 산출
- 로고 PNG → ANSI 배너로 변환 (이후 `/plan` 모드와 결합)
- 다이어그램 초안을 만들고 PR 설명에 첨부

파일 처리

생성된 이미지는 작업 디렉토리에 저장됩니다. 파일명은 보통 프롬프트에서 추론되거나 사용자가 명시적으로 지정합니다.

```
$imagegen logo.png 라는 이름으로 저장해줘.
크기는 1024x1024.
```

> 💡 **정보 —** 이미지 생성 결과는 작업 디렉토리에 저장되어 곧바로 `git add` 대상이 됩니다. 라이선스·저작권은 사용자 책임이며, 회사 자산으로 쓸 거라면 사용 정책을 미리 확인하세요.

> ✅ **TIP —** 이미지 생성과 코드 변경을 한 작업으로 묶고 싶다면 `/plan` 으로 먼저 단계를 짜는 패턴을 권합니다 — "1) 로고 생성 → 2) PNG를 SVG로 변환 → 3) README에 삽입" 같은 식으로요.

PART 4 · CHAPTER 1

01

Custom Slash Commands

자주 쓰는 프롬프트를
/명령어 한 줄로

개념

커스텀 슬래시 명령은 **마크다운 파일을 슬래시 명령으로 변환**하는 기능입니다. 자주 쓰는 프롬프트를 `~/.codex/prompts/`에 `.md`로 저장하면 그 파일명이 곧 명령어가 됩니다 — `review.md` → `/review` (단, 내장 명령과 충돌 시 내장 우선).

설정 단계

① 프롬프트 폴더 만들기

```
mkdir -p ~/.codex/prompts
```

중요한 점: **프로젝트 디렉토리가 아니라 사용자 홈**의 `~/.codex/prompts/`에 둬야 합니다.

▸ ② 마크다운 프롬프트 작성

```
# ~/.codex/prompts/draftpr.md
---
description: 브랜치 정리 후 draft PR 열기
argument-hint: [FILES=<경로>] [PR_TITLE="<제목>"]
---

이 작업을 위해 `dev/<feature_name>` 브랜치를 만들어줘.
파일이 지정되면 먼저 스테이지: $FILES.
스테이지된 변경을 명확한 메시지로 커밋해줘.
같은 브랜치에 draft PR을 열고, $PR_TITLE이 있으면 사용,
없으면 너 스스로 간결한 요약을 적어줘.
```

▸ ③ Codex 재시작

Codex를 재시작(또는 IDE 익스텐션 리로드)해야 새 프롬프트가 로드됩니다.

호출 방법

```
/                  # 슬래시 메뉴 열기
/prompts:draftpr       # 또는 직접 입력
/prompts:draftpr FILES=src/foo.ts PR_TITLE="버그 수정"
```

플레이스홀더 문법

구문	의미
`$1` ~ `$9`	공백으로 구분된 위치 인자
`$ARGUMENTS`	전체 인자 한 덩어리
`$FILE`, `$TICKET_ID` 등	이름 있는 인자, `KEY=value` 형태로 전달
`$$`	리터럴 `$` 한 글자

저장 위치 정리

- `~/.codex/prompts/` — 사용자 홈 (개인용)
- 저장소 안에 두는 것은 **지원되지 않음** — 그 용도로는 Skills 사용
- 마크다운(`.md`) 외 파일은 무시

⚠️ **주의 —** 사용자 정의 슬래시 명령은 공식적으로 **deprecated** 권고이며, 팀 공유·암시적 호출이 필요한 경우 OpenAI는 **Skills** 사용을 권장합니다. 다만 개인용 단축으로는 여전히 동작합니다.

💡 **정보 —** 팀 공유, 복잡한 분기 로직, 암시적 호출이 필요하다면 **Skills**로 옮기세요. 스킬은 저장소에 체크인되고, description 매칭으로 자동 발동되며, MCP·메타데이터·스크립트까지 묶을 수 있습니다.

PART 4 · CHAPTER 2

02

Skills

Agent Skills 설정과 사용

에이전트에게 새로운 전문 능력을
표준 형식으로 가르치기

스킬이란?

스킬은 Codex가 **특정 워크플로우를 신뢰성 있게 따르도록** 만드는 자족적 패키지입니다. 하나의 폴더에 지시문, 참조 자료, 선택적 스크립트가 함께 들어 있고, 오픈 표준인 agentskills.io 사양을 따릅니다.

스킬의 폴더 구조

```
my-skill/
├── SKILL.md          # 필수: 지시문 + 메타데이터 (name, description)
├── scripts/          # 선택: 실행 가능한 스크립트
├── references/       # 선택: 참조 문서
├── assets/           # 선택: 템플릿·리소스
└── agents/
    └── openai.yaml   # 선택: UI 메타·정책·의존성
```

최소 SKILL.md 예시

```
---
name: pr-reviewer
description: PR 변경분을 코드 품질·보안·테스트 관점에서 리뷰.
  PR 작성 직전이나 검토 요청 시 발동.
  단순 질의응답에는 발동하지 말 것.
---

다음 단계로 진행한다.
1. 변경된 파일 목록 추출
2. 각 변경에 대해 정확성·테스트 누락·보안 회귀 점검
3. 발견 사항을 우선순위와 함께 마크다운 표로 출력
```

두 가지 호출 방식

- **명시적 호출** · `/skills` 메뉴 또는 `$skill-name`
- **암시적 호출** · 사용자 요청이 description과 일치하면 Codex가 자동 선택

> 💡 **정보 —** 암시적 호출 정확도는 전적으로 **description의 품질**에 달려 있습니다. "언제 써야 하는지"와 "언제 쓰면 안 되는지"를 둘 다 명확히 적으세요.

스킬을 두는 위치 (스코프)

스코프	경로	용도
REPO	`$CWD/.agents/skills`	특정 디렉토리(서비스/모듈)에만 적용
REPO	`$REPO_ROOT/.agents/skills`	저장소 전체에 공통
USER	`$HOME/.agents/skills`	한 사용자가 모든 저장소에 사용
ADMIN	`/etc/codex/skills`	머신·컨테이너 단위 공통
SYSTEM	Codex 내장	OpenAI 기본 제공 (skill-creator, plan 등)

스킬 만들기 (가장 쉬운 방법)

```
$skill-creator
```

내장 크리에이터가 무엇을 하는 스킬인지, 언제 발동돼야 하는지, 스크립트를 포함할지를 차례로 묻고 폴더를 자동 생성합니다.

큐레이션된 스킬 설치

```
$skill-installer linear     # 예: Linear 통합 스킬
```

스킬 켜고 끄기

```
# ~/.codex/config.toml
[[skills.config]]
path = "/path/to/skill/SKILL.md"
enabled = false
```

변경 후에는 Codex를 재시작하세요.

모범 사례

- 스킬 하나는 **하나의 일**만 — 범위가 넓어지면 분해
- 가능하면 스크립트보다 지시문 위주 (디터미니즘이 꼭 필요할 때만 스크립트)
- 입력·출력을 명확히 적은 명령형 단계 사용
- description으로 트리거가 잘 잡히는지 실제 프롬프트로 테스트
- 저장소에 체크인할 스킬은 `.agents/skills` 에 — 팀 자동 공유

PART 4 · CHAPTER 3

03

MCP 서버 설정

전역과 프로젝트 스코프

Codex에
외부 도구의 손을 달기

MCP란?

MCP(Model Context Protocol)는 모델과 외부 도구·데이터 소스를 잇는 **표준 프로토콜**입니다. Codex는 STDIO와 Streamable HTTP 두 가지 트랜스포트를 모두 지원하며, 한 번 등록한 MCP 서버는 CLI와 IDE 익스텐션이 함께 사용합니다.

지원되는 트랜스포트

- **STDIO 서버** · 로컬 프로세스로 실행 (환경변수 전달 가능)
- **Streamable HTTP 서버** · URL로 접근. Bearer 토큰 또는 OAuth

설정 위치 — 전역 vs 프로젝트

스코프	경로	특징
전역	`~/.codex/config.toml`	모든 프로젝트에서 공통 사용
프로젝트	`.codex/config.toml`	신뢰된 프로젝트에서만 동작

(1) CLI로 추가하기

```
# STDIO 서버
codex mcp add <server-name> \
  --env VAR1=VALUE1 \
  -- <stdio server-command>

# 예: Context7 (개발자 문서)
codex mcp add context7 -- npx -y @upstash/context7-mcp

# 모든 MCP 명령 보기
codex mcp --help

# 활성 서버 목록 (TUI)
/mcp
```

(2) config.toml로 직접 작성

▸ STDIO 서버

```
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]

[mcp_servers.context7.env]
MY_ENV_VAR = "MY_ENV_VALUE"
```

▸ Streamable HTTP 서버 + Bearer 토큰

```
[mcp_servers.figma]
url = "https://mcp.figma.com/mcp"
bearer_token_env_var = "FIGMA_OAUTH_TOKEN"
http_headers = { "X-Figma-Region" = "us-east-1" }
```

▸ 도구 화이트/블랙리스트와 타임아웃

```
[mcp_servers.chrome_devtools]
url = "http://localhost:3000/mcp"
enabled_tools  = ["open", "screenshot"]
disabled_tools = ["screenshot"]
startup_timeout_sec = 20
tool_timeout_sec = 45
enabled = true
```

주요 옵션 정리

키	설명
`command / args / env / cwd`	STDIO 서버용
`url / bearer_token_env_var`	HTTP 서버용
`http_headers`	고정 헤더
`startup_timeout_sec` (기본 10)	서버 시작 대기
`tool_timeout_sec` (기본 60)	도구 실행 대기
`enabled`	삭제 없이 비활성화
`required = true`	이 서버 실패 시 Codex 시작 자체 실패

OAuth 로그인

```
codex mcp login <server-name>
```

자주 쓰이는 MCP 서버

- **OpenAI Docs MCP** · OpenAI 개발자 문서 검색
- **Context7** · 라이브러리 문서를 최신 상태로 제공
- **Figma** · 디자인 파일 접근
- **Playwright / Chrome DevTools** · 브라우저 제어와 스크린샷
- **Sentry / Linear / Jira MCP** · 이슈 트리아지
- **GitHub MCP** · PR·이슈 관리

> ✅ **TIP —** 프로젝트 단위 MCP는 **해당 프로젝트가 신뢰됨(trusted)**으로 표시되어야 동작합니다. 처음 들어가는 저장소는 trust 여부를 신중히 결정하세요.

PART 4 · CHAPTER 4

04

Subagents

전문 에이전트와 병렬 실행

큰 작업을 작게 쪼개
각자 맞는 모델·권한으로 위임

왜 서브에이전트인가?

큰 작업(코드베이스 탐색, 멀티 파일 리팩터링, 대규모 리뷰)을 단일 세션에 다 맡기면 컨텍스트가 오염되고("context pollution") 추적이 어렵습니다. 서브에이전트는 작업을 잘게 쪼개 **각각 별도 컨텍스트 윈도우**에서 돌리고 결과를 합칩니다.

현재 상태

- **2026년 3월부터 GA** — 몇 주간 preview 후 정식 기능
- 기본 활성화 — 별도 flag 불필요
- Codex 앱과 CLI에서 활동 가시화
- IDE 확장 가시화는 곧 추가 예정

두 가지 형태

형태	특징
기본 서브에이전트	Codex가 자동으로 사용 — 별도 정의 불필요
커스텀 에이전트	TOML 파일로 정의 — 자기 모델·권한·도구·지시문

커스텀 에이전트 정의

각 에이전트는 독립된 TOML 파일입니다.

▸ 저장 위치

스코프	경로
사용자	`~/.codex/agents/`
프로젝트	`.codex/agents/`

▸ 최소 예시

```
# ~/.codex/agents/reviewer.toml
name = "reviewer"
description = "PR 리뷰어 – 정확성·보안·테스트 누락 우선"

developer_instructions = """
오너십 있게 리뷰한다.
정확성, 보안, 동작 회귀, 테스트 커버리지 누락을 우선.
구체적 발견 사항 + 가능하면 재현 단계.
파일을 수정하지 말 것 – 발견 사항만 보고.
"""

sandbox_mode = "read-only"
```

주요 필드

필드	역할
`name`	에이전트 이름 (호출 키, 진실의 원천)
`description`	언제 사용할지
`developer_instructions`	이 에이전트의 시스템 프롬프트
`model`	이 에이전트가 쓸 모델 (예: gpt-5.4, gpt-5.3-codex-spark)
`model_reasoning_effort`	minimal / low / medium / high
`sandbox_mode`	read-only / workspace-write 등
`mcp_servers`	이 에이전트가 쓸 MCP 서버
`skills.config`	활성화할 스킬
`nickname_candidates`	UI 표시용 별명 목록 — 같은 에이전트 여러 개 띄울 때 구분용이

옵셔널 필드를 비우면 부모 세션에서 상속됩니다.

전형적인 3-에이전트 패턴

OpenAI 공식 문서가 권장하는 좁고 명확한 역할 분리.

```
# ~/.codex/agents/pr_explorer.toml
name = "pr_explorer"
description = "변경 제안 전 코드베이스를 읽기 전용으로 매핑·증거 수집"
model = "gpt-5.3-codex-spark"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"
developer_instructions = """
탐색 모드 유지. 실제 실행 경로를 추적하고 파일·심볼을 인용.
부모가 명시적으로 요청하지 않는 한 수정 제안은 하지 말 것.
"""
```

```
# ~/.codex/agents/reviewer.toml
name = "reviewer"
description = "정확성·보안·테스트 위험 검토"
model = "gpt-5.4"
sandbox_mode = "read-only"
```

```
# ~/.codex/agents/docs_researcher.toml
name = "docs_researcher"
description = "프레임워크/API 문서를 MCP로 조회"
mcp_servers = ["context7"]
sandbox_mode = "read-only"
```

호출 방법

▸ ① 명시적 호출

프롬프트에서 에이전트 이름을 직접 부릅니다.

```
설정 모달이 저장 안 되는 버그 조사.
browser_debugger로 재현하고
code_mapper가 책임 코드 경로 추적,
실패 원인이 명확해지면 ui_fixer가 최소 수정 구현.
```

▸ ② 자동 분배

"여러 에이전트로 나눠 실행"을 부탁하면 Codex가 알아서 분배합니다.

```
모듈별로 에이전트 하나씩 띄워서 동시에 진행:
- src/auth/의 인증 버그 수정
- src/api/orders.ts 에 검증 추가
- tests/integration/ 의 테스트 갱신
모두 끝나면 결과 통합 보고.
```

▸ ③ 슬래시 명령

실제 동작하는 명령은 두 가지로, 출력에 미묘한 차이가 있습니다.

명령	출력
`/agent list`	`Available agents/subagents:` — 내장 에이전트 (`default` · `explorer` · `worker`)와 커스텀 서브에이전트를 모두 표시
`/subagents list`	`Available subagents:` — 같은 목록을 서브에이전트 관점으로 표시

```
/agent list              # 활성·정의된 에이전트 목록
/agent <name>            # 그 에이전트 스레드로 전환·검사
/subagents list          # 별칭 – 위와 동일한 목록
```

공식 문서는 `/agent` (단수)를 표준으로 안내하며, `/subagents` 는 같은 동작의 별칭입니다.

전역 설정 — [agents] 섹션

```
# ~/.codex/config.toml
[agents]
max_threads = 6                       # 동시 실행 한도 (기본 6)
max_depth = 1                         # 중첩 깊이 (기본 1)
job_max_runtime_seconds = 300         # CSV 배치 워커 타임아웃
```

`max_depth = 1` 은 자식 에이전트만 허용하고 손자 에이전트는 차단합니다. 재귀 위임이 정말 필요한 경우만 깊이를 늘리세요.

오케스트레이션 동작

Codex는 다음을 자동으로 처리합니다.

- 새 서브에이전트 spawn
- follow-up 지시를 적절한 스레드로 라우팅
- 모든 결과를 기다린 후 통합 응답
- 완료된 에이전트 스레드 정리

승인과 샌드박스

서브에이전트는 부모의 샌드박스 정책을 상속합니다.

- 인터랙티브 CLI에서 비활성 스레드의 승인 요청도 메인에 표시
- 승인 오버레이가 소스 스레드 라벨을 보여줌 — `o` 키로 그 스레드로 이동해서 결정
- 비대화형 흐름에서 새 승인이 필요한 동작은 자동 실패 → 부모로 에러 전달
- 부모의 라이브 런타임 오버라이드(예: `/approvals` 변경)도 자식에 적용

좋은 커스텀 에이전트의 조건

- **좁고 명확한 역할** · 한 가지 일만 잘하는 에이전트
- **도구 표면이 역할에 맞음** · 리뷰어는 read-only, 실행자는 workspace-write
- **지시문이 인접 영역으로 새지 않게** · "X만 하고 Y는 부모가 요청할 때만"
- **모델 선택이 역할에 비례** · 탐색은 빠른 모델, 깊은 추론은 강한 모델

experimental — CSV 배치 처리

`spawn_agents_on_csv` 도구는 CSV의 각 행을 병렬 워커가 처리하게 합니다.

```
이 API 엔드포인트 CSV를 처리해줘.
각 행마다 {method}와 {path} 컬럼으로 통합 테스트를 생성.
```

각 워커는 `report_agent_job_result`를 정확히 한 번 호출하고, Codex가 결과를 메타데이터와 함께 CSV로 export합니다.

비용·관찰성

- **토큰은 스레드 수에 선형 비례** · 서브에이전트 N개 = 모델 추론 N회
- 각 에이전트는 자기 컨텍스트·도구·메모리
- `/status` 로 현재 활성 에이전트 확인

⚠️ **주의 —** 서브에이전트는 **토큰 사용량을 빠르게 늘립니다**. 정말 병렬화 가치가 있는 작업에만 쓰세요. 단일 흐름으로 충분한 작업은 단일로. `max_threads` 는 사고 방지 안전망일 뿐, 비용 통제의 본질은 사용자의 판단입니다.

💡 **정보 —** 서브에이전트는 정의 형식이 진화 중입니다 (TOML 외 마크다운 형태 요청도 활발). 지금은 TOML이 표준이니 위 패턴을 따르되, 향후 변경 가능성에 대비해 에이전트 정의를 너무 많이 쌓지 말고 정말 자주 쓰는 것만 유지하세요.

✅ **TIP —** 시작은 OpenAI 공식 문서의 **3-에이전트 패턴**(pr_explorer + reviewer + docs_researcher)으로. 이 셋만 잘 활용해도 PR 리뷰·코드 탐색·문서 확인 흐름의 80%가 자동화됩니다.

PART 5 · CHAPTER 1

01

Config 파일

Basic / Advanced / Reference

계층화된 config.toml로
모든 동작을 제어

설정 계층 (우선순위 높음 → 낮음)

1. CLI 플래그 (가장 강함)
2. 환경 변수
3. 프로젝트 `.codex/config.toml`
4. 사용자 `~/.codex/config.toml`
5. 시스템(엔터프라이즈) 정책
6. 내장 기본값

Config Basic — 가장 자주 쓰이는 설정

```
# ~/.codex/config.toml

# 기본 모델과 reasoning
model = "gpt-5.4"
model_reasoning_effort = "medium"

# 리뷰 전용 모델
review_model = "gpt-5.4"

# 기본 승인 모드
approval_mode = "auto"

# Auto Review
[review]
auto = true
preset = "uncommitted"
```

Config Advanced — 더 세밀한 제어

```
# MCP 서버
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp"]

# 스킬 활성/비활성
[[skills.config]]
path = "/path/to/skill/SKILL.md"
enabled = true

# 정책 규칙
[[rules]]
tool = "shell"
match = "^rm -rf"
action = "deny"

# TUI 커스터마이즈
[tui]
status_line = ["model", "context_usage", "git_branch"]
title = ["model", "session_id"]
```

Config Reference — 주요 키 정리

키	설명
`model`	기본 모델
`model_reasoning_effort`	minimal / low / medium / high
`review_model`	리뷰 전용 모델
`approval_mode`	read-only / auto / full-access
`sandbox_mode`	같은 값 + 에이전트 단위 적용 가능
`[review]`	Auto Review 설정
`[mcp_servers.*]`	MCP 서버 정의
`[[rules]]`	정책 규칙 (4부 05장)
`[[skills.config]]`	스킬 활성화 제어
`[agents]`	서브에이전트 한도(max_threads, max_depth)
`[tui]`	TUI 표시 항목

Sample Config — 빠른 시작용 템플릿

OpenAI는 공식 문서에 시작 템플릿을 제공합니다. 처음에는 그대로 가져와서 한두 항목만 바꾸면 됩니다.

일회성 오버라이드

```
codex --config model=gpt-5.4-mini --config approval_mode=read-only
```

`--config` 는 config.toml의 어떤 키든 임시로 덮어씁니다.

설정 진단

```
/debug-config
```

현재 적용된 설정이 어느 레이어(전역·프로젝트·CLI)에서 왔는지 추적해 보여 줍니다.

> 💡 **정보 —** 프로젝트 config를 저장소에 체크인하면 팀 전체가 같은 동작을 공유할 수 있습니다. 단 토큰·자격증명 같은 비밀은 절대 config에 직접 쓰지 말고 환경 변수(`$VAR_NAME`)로 처리하세요.

PART 5 · CHAPTER 2

02

Authentication

자격 증명 관리

누가 호출하는지,
어디서 비용이 나오는지

두 가지 인증 방식

방식	비용	적합한 사용처
ChatGPT 계정 로그인	구독 플랜에 포함	개인 사용, 팀 멤버 단위
OpenAI API 키	사용량당 과금	자동화, 서버, 조직 단위 통제

① ChatGPT 계정 로그인

처음 `codex`를 실행하면 OAuth 흐름으로 브라우저에서 ChatGPT 계정에 로그인합니다.

- Plus, Pro, Business, Edu, Enterprise 플랜에 Codex 사용량 포함
- 토큰은 `~/.codex/auth.json`에 저장

- 일정 기간 후 자동 갱신
- 다른 머신에서 새로 로그인하면 기존 세션은 유지

② API 키

```
export OPENAI_API_KEY="sk-..."
codex
```

▸ shell rc에 영구 설정

```
# ~/.zshrc 또는 ~/.bashrc
export OPENAI_API_KEY="sk-..."
```

▸ direnv로 프로젝트 단위 키 분리

```
# .envrc (저장소 루트)
export OPENAI_API_KEY="sk-..."
```

인증 전환

```
codex auth login          # 로그인 흐름 다시 시작
codex auth logout         # 자격증명 삭제
codex auth status         # 현재 인증 상태
```

OpenAI Organization과 Project

API 키는 organization 또는 project에 묶여 있습니다. 비용·쿼터 추적을 위해 적절히 분리해 두는 게 좋습니다.

```
export OPENAI_ORG_ID="org-..."
export OPENAI_PROJECT_ID="proj_..."
```

Vertex AI / Azure / 사내 게이트웨이

OpenAI 호환 API를 제공하는 다른 엔드포인트로도 라우팅할 수 있습니다.

```
# config.toml
api_base_url = "https://my-internal-gateway.example.com/v1"
api_key_env_var = "INTERNAL_KEY"
```

⚠️ **주의 —** 자격 증명은 절대 코드·config·로그에 직접 쓰지 마세요. 환경 변수로 주입하고, `.envrc` 같은 파일은 `.gitignore` 에 등록하세요. 노출이 의심되면 OpenAI 대시보드에서 즉시 키를 폐기·재발급하세요.

PART 5 · CHAPTER 3

03

Security

보안 모델 이해

CLI가 안전하게 동작하는
네 가지 층

네 가지 방어층

층	역할
① **Sandbox**	OS 수준 격리 (Seatbelt / Landlock)
② **Approval**	사용자 승인 흐름
③ **Policy / Rules**	도구·인자 매칭 기반 강제 정책
④ **Trust**	저장소·디렉토리 단위 신뢰 관리

① Sandbox — 가장 단단한 벽

- **macOS Seatbelt** · 파일 시스템·네트워크 정책을 커널 수준에서 강제

- **Linux Landlock** · 무력화 불가능한 파일 액세스 제한
- 모델이 우회 시도해도 OS가 차단
- Auto 모드의 워크스페이스 격리는 이걸로 보장

② Approval — 사용자 결정권

- 새로운 도구 호출은 매번 사용자 확인
- "같은 명령은 항상 허용" 옵션으로 마찰 줄임
- 위험한 명령(쓰기·실행·네트워크)은 더 보수적 처리

③ Policy / Rules — 강제 정책

4부 05장 참고. 사용자도 모델도 우회할 수 없는 deny/allow 규칙.

④ Trust — 신뢰된 디렉토리

처음 들어가는 폴더는 **신뢰되지 않은** 상태로 시작합니다. 이 상태에서는:

- 프로젝트 `.codex/config.toml` 미적용
- 프로젝트 MCP 서버 미실행
- 프로젝트 Custom Commands / Skills 비활성화
- AGENTS.md의 `@import` 차단

```
# 일회성 신뢰 (이번 세션만)
codex --skip-trust

# 영구 신뢰
codex trust add /path/to/repo
```

Cyber Safety

OpenAI는 Codex 사용 중 발견되는 보안 이슈에 대해 별도 가이드를 제공합니다.

- 모델이 의도치 않게 자격 증명을 출력하지 않도록 후처리
- 모델이 외부 명령을 통해 자기 자신의 권한을 확장하려 할 때 차단

- 프롬프트 인젝션 — 외부 콘텐츠가 모델을 조종하려는 시도 — 에 대한 가드레일

위협 모델

OpenAI 보안 문서는 다음을 위협 모델로 명시합니다.

- **악의적 코드** · 신뢰되지 않은 저장소가 Codex를 시켜 머신을 변형
- **탈취된 MCP 서버** · 외부 데이터를 통해 모델 행동 조종
- **의도치 않은 비용 폭증** · 무한 루프나 과도한 토큰 사용
- **자격증명 누출** · 키·토큰이 로그·출력으로 빠져나감

⚠️ **주의 —** Trust 모델을 끄면(`--skip-trust`) 그 저장소가 미리 심어 둔 악의적 MCP 서버나 hooks가 자동 시작될 수 있습니다. 오픈소스 저장소를 처음 클론할 때는 `.codex/` 폴더를 먼저 점검하는 습관을 들이세요.

PART 5 · CHAPTER 4

04

Enterprise 구성

Admin Setup과 Governance

조직 단위 통제와
일관된 정책 배포

Enterprise가 다루는 것

- **Admin Setup** · 조직 가입, SSO, SCIM, 좌석 관리
- **Governance** · 데이터 보존, 감사 로그, 접근 제어
- **Managed Configuration** · 머신 단위 강제 설정

Managed Configuration

IT 관리자가 사용자가 끌 수 없는 정책을 강제할 수 있습니다. 사용자 config.toml로도 덮어쓸 수 없습니다.

▶ 정책 파일 위치

OS	경로
macOS	`/Library/Application Support/Codex/managed-config.toml`
Linux	`/etc/codex/managed-config.toml`
Windows	`C:\ProgramData\Codex\managed-config.toml`

전형적인 엔터프라이즈 정책

```
# /etc/codex/managed-config.toml

# 인증 강제
[auth]
allowed_methods = ["api_key"]
required_org_id = "org-company"

# 모델 제한
allowed_models = ["gpt-5.4", "gpt-5.3-codex"]
default_model = "gpt-5.4"

# 네트워크 정책
[[rules]]
tool = "fetch"
match = "^https?://(?!.*\\.company\\.com)"
action = "deny"

# 위험 명령 차단
[[rules]]
tool = "shell"
match = "^(rm -rf|sudo|chmod 777)"
action = "deny"

# 텔레메트리 강제
[telemetry]
enabled = true
endpoint = "https://otel.company.internal/v1/traces"
log_prompts = false

# Trust 강제
[trust]
require_explicit_approval = true
disallow_skip_trust = true

# 실험 기능 차단
[experimental]
allow = false
```

강제할 만한 항목

- **인증 방식** · 개인 ChatGPT 로그인 금지, 회사 API 키만
- **모델 선택** · 검증된 모델만 허용
- **네트워크** · 외부 도메인 fetch 차단
- **Telemetry** · 사내 OTLP 수집기로 강제 (프롬프트 본문은 제외)
- **실험 기능** · 검증되지 않은 기능 일괄 차단
- **MCP 서버** · 화이트리스트로만 허용
- **Trust 우회 차단** · `--skip-trust` 비활성화

배포 도구

- **macOS** · MDM(Jamf, Intune) 프로파일
- **Windows** · Group Policy, Intune
- **Linux** · Ansible / Puppet / Chef

감사와 추적

- **OTLP 텔레메트리** · 모든 도구 호출, 모델 호출, 비용을 사내 수집기로
- **OpenAI 대시보드** · API 키 단위 사용량 추적
- **세션 로그** · 트랜스크립트가 자동 저장되어 사후 추적 가능

> 💡 **정보 —** 엔터프라이즈 환경에서는 **Trust 강제 + Managed Configuration + 사내 MCP만 허용**의 3종 세트가 표준입니다. 사용자가 임의로 외부 MCP를 추가하거나 신뢰되지 않은 저장소를 임시 허용하지 못하게 막습니다.

PART 5 · CHAPTER 5

05

Windows

환경 설정

윈도우에서 Codex CLI를
가장 잘 쓰는 법

현재 지원 상태

Codex CLI는 macOS와 Linux를 정식 지원하며, **Windows는 실험적**입니다. 가장 안정적인 방법은 **WSL(Windows Subsystem for Linux)**에서 실행하는 것입니다.

권장 — WSL2 + Ubuntu

```
# PowerShell에서
wsl --install -d Ubuntu

# Ubuntu가 뜨면
sudo apt update
sudo apt install -y nodejs npm
npm install -g @openai/codex
codex
```

왜 WSL을 권장하는가?

- 샌드박스(Landlock) 등 Linux 보안 기능을 그대로 사용
- Unix 도구 체인(git, grep, find, jq 등)이 자연스럽게 동작
- 대부분의 npm 패키지가 Linux를 우선 테스트
- Docker Desktop과의 통합도 매끄러움

네이티브 Windows 사용

꼭 필요하다면 PowerShell에서 직접 실행할 수 있습니다.

```
# PowerShell (관리자 모드)
npm install -g @openai/codex
codex
```

▸ 네이티브 모드의 한계

- 샌드박스 기능이 약함 — Approval에 더 의존
- 일부 셸 명령이 PowerShell 기준으로 변환되어야 함
- 경로 구분자(`\` vs `/`) 처리 이슈가 가끔 발생
- 일부 MCP 서버는 Windows 미지원

샌드박스에 디렉토리 추가 (Windows)

Windows 네이티브에서 워크스페이스 밖 폴더를 읽어야 할 때:

```
/sandbox-add-read-dir C:\absolute\directory\path
```

샌드박스 정책을 새로고침해 그 디렉토리에 읽기 권한을 부여합니다.

WSL에서 Windows 파일에 접근

```
# WSL 안에서 C: 드라이브 접근
cd /mnt/c/Users/<username>/projects/myapp
codex
```

WSL은 Windows 파일 시스템을 `/mnt/c/`로 마운트해 줍니다. 단 성능이 약간 떨어지므로, 가능하면 **WSL의 홈 디렉토리에 프로젝트를 두는 편**이 좋습니다.

윈도우 터미널 설정

- **Windows Terminal** 사용 권장 — 색상·글꼴 처리가 좋음
- Cascadia Code, JetBrains Mono 같은 모노스페이스 폰트
- UTF-8 설정 — 한국어·이모지 깨짐 방지

> 💡 **정보 —** VS Code의 "Remote - WSL" 익스텐션을 함께 쓰면 WSL 안의 Codex 세션을 IDE처럼 자연스럽게 관리할 수 있습니다. `code .` 로 WSL 디렉토리를 열면 모든 환경(Codex, git, Node)이 그대로 연결됩니다.

PART 6 · CHAPTER 1

01

Pricing과 사용량

어디서 비용이 나오는지

무료 한도 / 구독 플랜 /
API 과금

인증 방식별 비용 모델

인증	비용 구조	적합한 사용처
ChatGPT 구독	플랜 정액 + 일/주 사용량 한도	개인·팀 멤버 단위
API 키	입력·출력 토큰당 과금	자동화·서버·CI

Codex가 포함된 ChatGPT 플랜

- ChatGPT **Plus** — 가장 저렴한 시작점
- ChatGPT **Pro** — 개인 헤비 유저
- ChatGPT **Business** — 소규모 팀
- ChatGPT **Edu** — 교육 기관

- ChatGPT **Enterprise** — 대규모 조직

현재 사용량 확인

```
/status
```

활성 모델, 컨텍스트 사용량, 누적 토큰을 한 번에 보여 줍니다.

한도 도달 시 동작

- 구독 플랜 — 사용량 한도에 도달하면 일정 시간 후 자동 회복
- API 키 — 청구가 누적되며, 사용량 한도(spend cap)를 OpenAI 대시보드에서 설정 가능
- 한도 초과 빈도가 잦으면 — 더 큰 플랜이나 API 키 전환 검토

비용 절감 패턴

패턴	효과
탐색은 `gpt-5.4-mini`, 작업은 `gpt-5.4`	토큰 50%+ 절감
reasoning은 기본 `medium`, 어려울 때만 `high`	응답 시간·토큰 모두 감소
긴 AGENTS.md → 핵심만	매 호출 입력 토큰 감소
`/compact` 적극 활용	컨텍스트 청구 토큰 회수
Plan 모드로 검증 후 실행	실패한 작업 재시도 비용 회피
Auto Review를 매번 → PR 직전만	중복 리뷰 비용 감소

비용 모니터링

- **OpenAI 플랫폼 대시보드** · 일자별·프로젝트별 비용
- **Spend cap 설정** · 월 한도 초과 시 자동 차단
- **OTLP 텔레메트리** · 사내 모니터링 시스템에 토큰 사용량 전송

✅ **TIP** — 정확한 가격·한도는 시점에 따라 변하므로, 확정된 수치는 항상 **developers.openai.com/codex/pricing**에서 확인하세요. 큰 자동화를 도입하기 전에는 작은 규모로 1-2주 운영하면서 실제 토큰 패턴을 측정하는 게 좋습니다.

PART 6 · CHAPTER 2

02

Non-interactive Mode 와 Codex SDK

사람 없이 도는
Codex

Non-interactive Mode

인터랙티브 TUI 없이 한 번 실행하고 결과만 받는 모드입니다. CI, cron, 스크립트에 적합합니다.

```
codex exec "이 저장소의 README를 한 페이지로 요약해줘"
codex exec --full-auto "테스트 실패를 모두 고쳐줘"
codex exec resume --last "어제 작업 마저 진행해줘"
```

주요 옵션

옵션	설명
`codex exec "<prompt>"`	1회 실행
`--full-auto`	모든 도구 자동 승인
`--output-format json`	구조화 출력 (스크립트 파싱용)
`--cd <dir>`	작업 디렉토리
`resume --last`	이전 세션 이어서

전형적인 자동화 패턴

```
# 1. 매일 아침 코드베이스 헬스체크
0 9 * * * cd /repo && codex exec --full-auto \
  "어제 이후 변경된 파일을 검토하고 위험 신호를 알려줘" \
  > /var/log/codex-daily.log

# 2. 커밋 직전 자동 리뷰
codex exec --full-auto "/review uncommitted" \
  --output-format json | jq '.findings[]' \
  | tee review.log
```

Codex SDK

Codex SDK는 코드에서 직접 Codex를 호출하는 라이브러리입니다. `codex exec`를 셸에서 부르는 대신, 프로그램 내부에서 호출해 결과를 처리합니다.

```
// TypeScript / Node.js
import { Codex } from "@openai/codex";

const codex = new Codex({ model: "gpt-5.4" });

const result = await codex.run({
  prompt: "이 함수 리팩터링해줘: ...",
  cwd: "./apps/web",
  approvalMode: "auto",
});

console.log(result.output);
console.log(result.fileDiffs);
```

SDK 활용 사례

- 커스텀 IDE 익스텐션
- 사내 개발자 포털에 코드 분석 기능 통합
- 대량 마이그레이션 스크립트(여러 저장소를 순회하며 동일 작업 적용)
- 이슈 트리아지 봇

관련 자동화 인터페이스

인터페이스	용도
Non-interactive	셸 스크립트, cron, 간단한 CI
Codex SDK	다른 프로그램 안에서 직접 호출
App Server	HTTP API로 Codex를 노출 (다음 장)
GitHub Action	GitHub Actions 워크플로우 (4부 06장)
MCP Server	다른 LLM 도구에 Codex를 도구로 제공

> 💡 **정보 —** 자동화 비용을 통제하려면 **Spend cap + 보수적 모델 선택 + Rules로 명령 제한**의 3종 세트가 효과적입니다. 처음에는 작은 cron으로 시작해 패턴을 확인한 뒤 점진적으로 확장하세요.

PART 6 · CHAPTER 3

03

App Server / MCP Server

Codex를
다른 시스템에 노출하기

두 가지 서버 모드

모드	역할	사용처
App Server	HTTP API로 Codex 노출	웹 UI, 사내 개발자 포털
MCP Server	Codex를 MCP 도구로 노출	다른 LLM 클라이언트가 Codex를 도구로 사용

App Server

Codex를 백그라운드에서 띄우고 HTTP API로 호출할 수 있습니다.

```
codex serve --port 3000 --auth-token mysecret
```

▸ API 호출 예

```
curl -X POST http://localhost:3000/v1/run \
  -H "Authorization: Bearer mysecret" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "이 파일을 리뷰해줘",
    "cwd": "/path/to/repo",
    "approval_mode": "read-only"
  }'
```

▸ 적합한 사용처

- 사내 개발자 포털 — 모든 개발자가 같은 인스턴스 공유
- VS Code 익스텐션의 백엔드
- Slack 봇 — 채널 메시지에 응답하는 코딩 어시스턴트
- 여러 사용자가 같은 워크스페이스를 비동기로 사용

MCP Server

이번에는 반대로, Codex 자체를 **MCP 도구**로 만들어 다른 LLM 클라이언트(다른 Codex, Claude Desktop, 사내 LLM 도구 등)가 Codex를 호출할 수 있게 합니다.

```
codex mcp serve
```

▸ 다른 클라이언트에서 등록

```
# 다른 Codex 인스턴스의 config.toml
[mcp_servers.codex-helper]
command = "codex"
args = ["mcp", "serve"]
```

MCP Server의 활용

- **중첩 서브에이전트** · 한 Codex가 다른 Codex 인스턴스를 도구로 호출
- **크로스 LLM 통합** · Claude Desktop이 Codex를 도구로 호출

- **전문화된 인스턴스** · "보안 리뷰 전용 Codex"를 하나의 도구로 노출

Server 모드의 보안 고려

- **인증 필수** · Bearer 토큰, mTLS 등으로 접근 제한
- **네트워크 격리** · 외부 노출 시 VPN·방화벽 뒤에
- **워크스페이스 분리** · 사용자별 cwd 격리
- **감사 로그** · 모든 호출 기록 (OTLP 텔레메트리)
- **Spend cap** · API 키 단위 한도

> ⚠️ **주의 —** App Server를 외부망에 노출하는 건 매우 위험합니다 — 인증을 뚫리면 공격자가 Codex의 모든 권한(파일 쓰기, 셸 실행)을 얻습니다. 반드시 사내망 안에서, 그것도 추가 인증층 뒤에 두세요. 가능하면 사용자별 격리된 컨테이너에서 실행하는 것이 가장 안전합니다.

PART 6 · CHAPTER 4

04

Troubleshooting

자주 만나는 문제와
빠른 해결

설치 / 실행

"command not found: codex"

- `npm root -g` 로 글로벌 경로 확인 → 그 경로의 `bin` 이 PATH에 있는지
- nvm 사용 중이면 nvm 환경 활성화 후 다시 설치
- Homebrew 환경에서는 `brew doctor` 로 PATH 확인

버전 미스매치

```
codex --version
npm i -g @openai/codex@latest
```

인증

▸ 로그인 브라우저가 안 열림

- SSH 환경 → 디바이스 코드 흐름 또는 API 키 방식
- 방화벽 정책으로 로컬 콜백 차단 → API 키로 전환
- `codex auth logout && codex auth login`으로 재시도

▸ "Insufficient quota"

- OpenAI 대시보드에서 Spend cap, billing 상태 확인
- 구독 플랜의 사용량 한도 초과인지 확인 (몇 시간 후 회복)
- 모델을 `gpt-5.4-mini`로 임시 전환

도구 / 권한

▸ "This folder is not trusted"

5부 03장의 Trust 모델 참고.

```
codex --skip-trust    # 일회성 우회
codex trust add .     # 영구 신뢰
```

▸ 특정 셸 명령이 매번 차단됨

```
# Rules에 allow 추가
[[rules]]
tool = "shell"
match = "^pnpm test"
action = "allow"
```

▸ Sandbox가 너무 빡빡함

```
codex --sandbox workspace-write
# 또는 잠시만
/permissions
```

성능 / 비용

응답이 너무 느림

- 모델을 `gpt-5.3-codex` 나 `gpt-5.4-mini` 로
- reasoning을 `medium` 이나 `low` 로
- AGENTS.md 길이 점검 (불필요한 import 제거)
- `/compact` 로 컨텍스트 정리

비용이 갑자기 늘어남

- `/status` 로 reasoning 단계 확인 — `high` 가 켜진 채로 잊혀진 경우가 흔함
- Auto Review가 매 변경마다 도는지 확인
- 서브에이전트가 의도치 않게 자주 발동되는지 확인

MCP / 확장

MCP 서버가 시작 안 됨

- `/mcp` 로 활성 상태와 에러 메시지 확인
- `codex --debug` 로 상세 로그
- STDIO 서버는 `command` 가 PATH에 있는지
- HTTP 서버는 URL과 인증 토큰 확인

스킬이 트리거되지 않음

- `/skills` 로 enable 상태 확인
- SKILL.md의 description 강화 (언제 발동, 언제 발동 금지)
- Codex 재시작

Custom Slash Command가 안 보임

- `~/.codex/prompts/` 위치 확인 (프로젝트 폴더 아님)
- `.md` 확장자인지 확인
- Codex 재시작

디버그 모드

```
codex --debug
# 또는
CODEX_DEBUG=1 codex
```

모든 도구 호출, MCP 메시지, 모델 요청·응답이 로그에 출력됩니다.

> 💡 **정보 —** 재현되는 최소 시나리오를 만들어 GitHub **`openai/codex`** 이슈로 신고하면 빠르게 해결됩니다. **`--debug`** 출력과 OS·Node·Codex 버전을 함께 첨부하세요.

PART 6 · CHAPTER 5

05

함께 쓸 때의 주의사항

Skills · MCP · Custom Commands · Rules

① 공통 — 우선순위와 충돌

- 이름이 같은 **커스텀 에이전트**는 내장 에이전트(예: `explorer`)를 덮어씀
- **커스텀 슬래시 명령**은 내장 명령(예: `/review`)과 이름 겹치면 내장이 우선
- **같은 이름 스킬**이 여러 스코프(USER vs REPO)에 있으면 둘 다 노출 — 이름 모호하면 잘못 선택 가능
- Rules는 모델보다 위에서 동작하므로 자연어 우회 시도도 차단

② Skills 주의사항

- **description 품질이 정확도** · 암시적 호출은 description 매칭에 의존
- **스코프 폭주** · 한 스킬에 여러 일을 욱여넣으면 트리거 정확도 하락
- **스크립트 의존성** · 머신·런타임에 따라 깨질 수 있음. 가능하면 지시문 위주

- **변경 후 갱신** · 보통 자동 감지되지만 안 보이면 Codex 재시작
- **Trust되지 않은 폴더에서 미동작** · 프로젝트 스킬은 trusted 표시 필수

③ MCP 주의사항

- **프롬프트 인젝션 표면 확장** · 외부 데이터로 모델을 조종하려는 시도 가능 — 신뢰 가능한 서버만 사용
- **도구 화이트리스트** · 서버가 노출하는 도구가 너무 많으면 `enabled_tools`로 제한
- **타임아웃 조정** · 느린 서버는 `tool_timeout_sec`를 충분히
- **자격 증명** · 환경변수로 주입, config에 직접 쓰지 말 것
- **required = true**는 신중히 — 서버 실패가 곧 Codex 시작 실패

④ Custom Slash Commands 주의사항

- **Deprecated 권고** · 새로 만들 거라면 Skills를 우선 검토
- **저장소에 체크인 불가** · 사용자 홈이라 팀과 공유 안 됨
- **비-마크다운 파일은 무시** · `.md`가 아니면 인식 안 됨
- **변경 시 재시작** · 프롬프트 파일을 추가·수정하면 Codex 재시작
- **인자 이스케이프** · 공백 포함 값은 따옴표로 (`TITLE="버그 수정"`)

⑤ Rules 주의사항

- **너무 많은 deny는 마찰** · 모델이 우회 경로를 찾느라 컨텍스트 낭비
- **정규식 신중하게** · 너무 느슨하면 의도치 않게 안전 명령까지 차단
- **allow는 보수적으로** · 자동 승인되는 패턴은 한 번 더 검토
- **엔터프라이즈 정책 우선** · Managed Configuration의 deny는 사용자가 못 풂

⑥ 서로 결합할 때

- 커스텀 에이전트가 자체 `mcp_servers` · `skills.config`를 가질 수 있음 → 의도된 격리인지 점검
- 같은 스킬을 부모와 서브에이전트가 동시에 부르면 부작용 누적 가능 — 파일 쓰는 스킬은 락이나 타임스탬프 방어

- 커스텀 슬래시 명령에서 서브에이전트를 트리거하는 경우, 결과를 사람이 합쳐야 함 — 자동화하려면 스킬로

> ✅ **TIP —** 새 워크플로우를 만들 때는 **Skills 1순위, Custom Commands는 임시 단축, Rules는 안전 가드레일** 순서로 고르면 함정을 가장 적게 밟습니다. MCP는 외부 시스템 통합이 꼭 필요할 때만 추가하세요.

PART 6 · CHAPTER 6 · 끝맺으며

06

마무리

Codex CLI의 강점과
활용 추천

다시 강조하는 강점

- **터미널 네이티브 + Rust** · 빠른 시작, 가벼운 메모리. SSH·tmux·dotfiles와 자연스럽게 어울림
- **오픈소스** · 동작이 투명해 디버깅·기여 가능
- **4겹 보안 모델** · Sandbox + Approval + Rules + Trust로 안전 기본값
- **공유되는 설정** · CLI에서 만든 MCP·Skills·AGENTS.md·인증이 IDE·App과 그대로 호환
- **강력한 자동화** · Non-interactive, SDK, App Server, GitHub Action으로 어디든 통합
- **엔터프라이즈 친화** · Managed Configuration으로 조직 단위 통제

활용 시나리오 추천

- **1인 개발자** · AGENTS.md + `/plan` + `/review` 루틴화로 PR 품질 향상

- **팀** · 저장소 루트에 `.codex/` (skills·agents·config)를 체크인해 워크플로우 공유
- **오픈소스 메인테이너** · GitHub Action으로 이슈 트리아지·PR 리뷰·릴리스 노트 자동화
- **SRE/플랫폼** · MCP로 Sentry·Linear·GitHub 연결 + 인시던트 트리아지
- **학습·탐색** · 새 코드베이스에 들어가자마자 Read Only로 `/plan`해 구조부터 그리기
- **엔터프라이즈** · Managed Configuration + 사내 OTLP + 사내 MCP로 완전 통제 환경

운영 원칙 세 가지

1. **승인 모드를 의식적으로** — Read Only / Auto / Full Access를 작업 단위로 전환. 기본은 Auto, 위험할 땐 Read Only로 후퇴
2. **반복은 Skills로 묶기** — 같은 흐름을 두 번 이상 했다면 스킬로 패키징. 팀과 공유
3. **병렬은 정말 필요할 때만** — 서브에이전트는 강력하지만 비용도 빠르게 늘어남. 단일 흐름으로 충분한 작업은 단일로

마지막 한 마디

Codex CLI는 "터미널에서 같이 일하는 동료"이면서 동시에 "조직 전체에 일관된 자동화를 배포하는 플랫폼"입니다. 위의 세 가지 원칙만 지키면 비용·안정성·생산성이 모두 균형을 잡습니다.

💡 정보 — 참고 문서

- 공식 사이트 — developers.openai.com/codex
- CLI Overview — developers.openai.com/codex/cli
- Slash commands — developers.openai.com/codex/cli/slash-commands
- MCP — developers.openai.com/codex/mcp
- Skills — developers.openai.com/codex/skills
- AGENTS.md — developers.openai.com/codex/guides/agents-md
- Rules — developers.openai.com/codex/rules
- Config Reference — developers.openai.com/codex/config-reference
- Codex SDK — developers.openai.com/codex/sdk
- GitHub Action — developers.openai.com/codex/github-action
- 오픈소스 — github.com/openai/codex

Codex CLI 완벽 가이드

설치부터 엔터프라이즈 운영까지

OpenAI Developer Documentation 기반

참조 · developers.openai.com/codex

이 교재는 Codex CLI 공식 문서를 학습 목적으로 정리한 자료입니다.

실제 명령·옵션은 Codex CLI 버전에 따라 달라질 수 있습니다.